

UNIT IV

PYTHON LIBRARIES FOR DATA WRANGLING

Course Objective – To utilize the Python libraries for Data Wrangling.

Course Outcome – Use the Python Libraries for Data Wrangling.

Bloom's Taxonomy Level – K1, K2, K3

PART A

S.No.	Question	BTL								
1.	What is data wrangling? Data wrangling (or data munging) is the process of removing errors and combining complex data sets to make them more accessible and easier to analyze. It consists of reorganizing, transforming and mapping data from one "raw" form into another in order to make it more usable and valuable for a variety of downstream uses including analytics.	K2								
2.	What is NumPy in python used for? NumPy is a fundamental library for numerical and scientific computing in Python, providing efficient data structures and functions for a wide range of mathematical operations. It plays a crucial role in the Python ecosystem for scientific computing, machine learning, data analysis, etc.	K2								
3.	Write a python program to show the different ways to create an array. import numpy as np arr1 = np.zeros(5) # creating an array of zeros arr2 = np.ones(5) # creating an array of ones arr3 = np.array([1, 4, 2, 5, 3]) # Creating an integer array arr4 = np.arange(0, 20, 2) # Creating an array filled with a linear sequence arr5 = np.linspace(0, 1, 5) # Creating an array of five values evenly spaced between 0 and 1 arr6 = np.random.rand(3) # Creating an array using random values	K3								
4.	Write python code to create 1D, 2D and 3D numpy arrays. import numpy as np array_1d = np.array([1, 2, 3, 4, 5]) array_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) array_3d = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])	K3								
5.	Write the output for the following numpy codes. <table><tr><td>(i) np.array([3,14,4,2,3])</td><td>array([3, 14, 4, 2, 3])</td></tr><tr><td>(ii) np.array([1,2,3,4],dtype='float32')</td><td>array([1., 2., 3., 4.], dtype=float32)</td></tr><tr><td>(iii) np.arange(2, 11).reshape((3, 3))</td><td>array([[2, 3, 4], [5, 6, 7], [8, 9, 10]])</td></tr><tr><td>(iv) np.zeros(6, dtype = int)</td><td>array([0, 0, 0, 0, 0, 0])</td></tr></table>	(i) np.array([3,14,4,2,3])	array([3, 14, 4, 2, 3])	(ii) np.array([1,2,3,4],dtype='float32')	array([1., 2., 3., 4.], dtype=float32)	(iii) np.arange(2, 11).reshape((3, 3))	array([[2, 3, 4], [5, 6, 7], [8, 9, 10]])	(iv) np.zeros(6, dtype = int)	array([0, 0, 0, 0, 0, 0])	K3
(i) np.array([3,14,4,2,3])	array([3, 14, 4, 2, 3])									
(ii) np.array([1,2,3,4],dtype='float32')	array([1., 2., 3., 4.], dtype=float32)									
(iii) np.arange(2, 11).reshape((3, 3))	array([[2, 3, 4], [5, 6, 7], [8, 9, 10]])									
(iv) np.zeros(6, dtype = int)	array([0, 0, 0, 0, 0, 0])									

	(v) np.ones((3, 5), dtype = float) (vi) np.arange(0, 20, 20) (vii) np.linspace(0, 1, 5) (viii) np.random.randint(5, size=8)	array([[1., 1., 1., 1., 1.], [1., 1., 1., 1., 1.], [1., 1., 1., 1., 1.]]) array([0]) array([0., 0.25, 0.5 , 0.75, 1.]) array([4, 0, 1, 4, 3, 2, 3, 0])	
6.	What are ufuncs? Mention few ufuncs used with numpy arrays. Universal Functions, or ufuncs, in NumPy are functions that operate element-wise on arrays, meaning they perform element-wise operations on each element of the input array. Ufuncs are a key component of NumPy, providing fast and efficient operations on arrays without the need for explicit looping. They are implemented in compiled C code, making them highly efficient. Few ufuncs: Mathematical Operations: np.add(x1, x2), np.multiply(x1, x2) Trigonometric Functions: np.sin(x), np.cos(x) Exponential and Logarithmic Functions: np.exp(x), np.log(x), np.log10(x) Statistical Functions: np.mean(x), np.std(x) Comparison Functions: np.equal(x1, x2), np.greater(x1, x2) Miscellaneous Functions: np.sqrt(x), np.absolute(x), np.power(x, n)		K2
7.	Define computations on arrays. Computations on arrays in NumPy involve performing mathematical or statistical operations on entire arrays or elements of arrays in an element-wise manner. Computations on arrays in NumPy are fundamental for scientific computing, data analysis, and machine learning tasks. They provide a concise and efficient way to work with large datasets and perform complex operations. This is achieved through the use of ufuncs (universal functions) in NumPy, which operate on arrays element by element. Eg. Element-wise Operations:, broadcasting, aggregations, Logical and Comparison Operations, Mathematical and Statistical Functions:		K1
8.	Write the list of aggregate functions of numpy. np.sum - Compute sum of elements np.prod - Compute product of elements np.mean - Compute median of elements np.std - Compute standard deviation np.var - Compute variance np.min / np.max - Find minimum / maximum value np.argmin / np.argmax - Find index of minimum / minimum value np.median - Compute median of elements np.percentile - Compute rank-based statistics of elements np.any / np.all - Evaluate whether any / all elements are true		K2
9.	What are nan functions in python? Give few examples.		K2

	NaN (Not a Number) functions in NumPy refer to functions that are designed to work with arrays containing NaN values. NaN is a special floating-point value that represents undefined or unrepresentable results in numerical computations. In the context of NumPy, NaN functions are functions that are aware of NaN values and provide mechanisms for handling them in array operations. Eg. <code>np.isnan()</code>, <code>np.nanmin()</code>, <code>np.nansum()</code>:	
10.	What is broadcasting? Broadcasting is simply a set of rules for applying binary ufuncs (addition, subtraction, multiplication, etc.) on arrays of different sizes. It is a powerful feature in NumPy that allows operations to be performed on arrays of different shapes and sizes. It enables NumPy to work with arrays of different shapes and dimensions in a way that makes sense mathematically, without the need for explicit replication or looping. Eg. <code>a = np.array([0, 1, 2])</code> <code>b = 5</code> <code>print(a + b)</code> OUTPUT: <code>([5, 6, 7])</code>	K2
11.	Mention how masking operation is used to access the elements of numpy arrays. Masking is a technique used to access or modify elements in an array based on a specified condition. A mask is a boolean array that has the same shape as the original array, where each element of the boolean array indicates whether the corresponding element in the original array satisfies a certain condition. Eg. <code>>>> arr1 = np.array([[5, 0, 3, 3], [7, 9, 3, 5], [2, 4, 7, 6]])</code> <code>>>> arr1[arr1 < 5]</code> <code>([0, 3, 3, 3, 2, 4])</code> # one-dimensional array filled with all the values that meet the (values < 5) condition is returned	K2
12.	What is fancy indexing? Fancy indexing in NumPy refers to indexing arrays using arrays of indices or boolean masks rather than simple scalars or slices. It allows to pass arrays of indices in place of single scalars and to very quickly access and modify complicated subsets of an array's values. It provides a powerful and flexible way to access or modify elements in an array based on sets of indices. Eg. <code>>>> arr = np.array([10, 20, 30, 40, 50])</code> <code>>>> ind = np.array([1, 3, 4])</code> <code>>>> arr[ind]</code> <code>([20, 40, 50])</code>	K2
13.	Write about the need for Numpy's structured arrays. NumPy's structured arrays are a powerful feature that extends the	K2

	functionality of traditional NumPy arrays by allowing the creation of arrays with compound data types, similar to structured records or database tables. These structured arrays are also known as "record arrays" or "structured data types". They provide efficient storage for compound, heterogeneous data. The need for NumPy's structured arrays arises from several requirements in scientific computing, data analysis, and numerical simulations	
14.	Write short note on pandas. Pandas is an open-source data manipulation and analysis library for Python. It provides easy-to-use data structures and functions needed to manipulate structured data seamlessly. Some key features and aspects of Pandas include: • DataFrame and Series: • Data Input/Output (as csv, Excel, SQL databases, etc.) • Data Cleaning and Preprocessing: • Indexing and Selection • Grouping and Aggregation • Merge and Join Operations • Visualization	K2
15.	Define series object. Series is a fundamental data structure used in Pandas for handling and manipulating structured data. A Pandas Series is a one-dimensional labeled array capable of holding any data type. It is similar to a column in a spreadsheet or a single variable in statistics. A Series consists of data and index.	K2
16.	What is DataFrame? DataFrame is a fundamental data structure used in Pandas for handling and manipulating structured data. A Pandas DataFrame is a two-dimensional, tabular data structure resembling a spreadsheet or SQL table. It is composed of rows and columns, each with its own index. A DataFrame can be thought of as a collection of Series, where each Series represents a column. A Series consists of data, index and columns.	K2
17.	How can a Pandas data frame be constructed? <pre>import pandas as pd # Creating a DataFrame from a dictionary data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 22], 'City': ['Chennai', 'Mumbai', 'Delhi']} df1 = pd.DataFrame(data) OUTPUT: Name Age City 0 Alice 25 Chennai 1 Bob 30 Mumbai</pre>	K3

	2 Charlie 22 Delhi																
18.	What is data indexing and selection? Data indexing and selection refer to the process of accessing, retrieving, and manipulating specific elements or subsets of data within a data structure. In the context of Pandas, which is commonly used for data analysis, indexing and selection are crucial operations for working with Series and DataFrame objects.	K2															
19.	What are indexers? Indexers are attributes provided by to explicitly expose certain indexing schemes. They are not functional methods, but attributes that expose a particular slicing interface to the data in the Series. They represent whether the index to access a data element is used implicitly or explicitly or a hybrid of both. The indexers are: loc, iloc and ix.	K2															
20.	What is missing data? How can missing data be handled in python? Missing data, also known as "missing values" or "null values," refers to the absence of information in a dataset for a particular variable or observation. In Python, missing data is often represented by the special value NaN (Not a Number) in numerical arrays or None in object arrays. Handling missing data is a critical step in data preprocessing because it can impact the accuracy and reliability of statistical analyses and machine learning models. Strategies that can be employed to deal with missing data in Python: • Dropping Missing Values • Replace missing values with a specific value • Forward or backward fill	K2															
21.	Differentiate NaN and None in pandas. <table border="1"> <thead> <tr> <th>Aspect</th><th>NaN</th><th>None</th></tr> </thead> <tbody> <tr> <td>Representation</td><td>NaN is a special floating-point value that represents missing or undefined data in numerical arrays</td><td>None is a built-in object representing the absence of a value or a null pointer.</td></tr> <tr> <td>Purpose</td><td>Commonly used for missing numerical values</td><td>Used in object arrays in Pandas to represent missing or undefined data</td></tr> <tr> <td>Data type</td><td>Float type</td><td>Object type</td></tr> <tr> <td>Usage</td><td>Used in numeric arrays when a value is not available or applicable.</td><td>Can be used in both numeric and non-numeric arrays.</td></tr> </tbody> </table>	Aspect	NaN	None	Representation	NaN is a special floating-point value that represents missing or undefined data in numerical arrays	None is a built-in object representing the absence of a value or a null pointer.	Purpose	Commonly used for missing numerical values	Used in object arrays in Pandas to represent missing or undefined data	Data type	Float type	Object type	Usage	Used in numeric arrays when a value is not available or applicable.	Can be used in both numeric and non-numeric arrays.	K2
Aspect	NaN	None															
Representation	NaN is a special floating-point value that represents missing or undefined data in numerical arrays	None is a built-in object representing the absence of a value or a null pointer.															
Purpose	Commonly used for missing numerical values	Used in object arrays in Pandas to represent missing or undefined data															
Data type	Float type	Object type															
Usage	Used in numeric arrays when a value is not available or applicable.	Can be used in both numeric and non-numeric arrays.															
22.	Define Hierarchical indexing. Hierarchical indexing (or multi-level indexing), is a feature in Pandas that allows to have multiple index levels on one or more axes of a Pandas	K2															

	DataFrame or Series. This enables the representation of higher-dimensional data in a two-dimensional tabular structure. With hierarchical indexing, we can create more complex data structures, perform advanced data manipulations, and represent multi-dimensional data in a more intuitive way.	
23.	What is the use of groupby? The groupby function in Pandas is a powerful tool for grouping data based on one or more criteria and then applying a function to each group independently. It is commonly used for data analysis tasks such as aggregation, transformation, and filtering. The most important operations made available by a GroupBy are aggregate, filter, transform, and apply.	K2
24.	What is pivot table? The pivot table takes simple column-wise data as input, and groups the entries into a two-dimensional table that provides a multidimensional summarization of the data. It allows users to transform and reshape a dataset by aggregating and summarizing data based on one or more key variables.	K2

PART B

S.No	Question	BTL
1.	Briefly explain the basics of numpy arrays with examples.	K2
2.	What is broadcasting in numpy? Explain broadcasting and the rules for broadcasting with relevant examples.	K2
3.	Describe masking and fancy indexing with suitable examples.	K2
4.	Explain structured data in numpy arrays.	K1
5.	Explain how operating on data can be performed in Pandas.	K1
6.	Write in detail about the Hierarchical indexing and pivot tables.	K2
7.	Explain combining datasets using concat, append, merge and join functions.	K2

UNIT V

DATA VISUALIZATION

Course Objective – To present and interpret data using visualization libraries in Python.

Course Outcome – Apply visualization libraries in Python to interpret and explore data.

Bloom's Taxonomy Level – K1, K2, K3

PART A

S.No.	Question	BTL
1.	What is the purpose of matplotlib? Matplotlib is a tool for visualization in Python. It is a multiplatform data visualization library built on NumPy arrays, and designed to work with the broader SciPy stack. It is a versatile and powerful plotting library that plays a crucial role in the Python scientific computing ecosystem, providing users with the tools they need to visualize and communicate data effectively.	K2
2.	Write a Python code to draw a simple line plot using matplotlib. <pre>import matplotlib.pyplot as plt import numpy as np fig = plt.figure() x = np.linspace(0, 10, 100) y = np.sin(x) plt.plot(x, y) plt.plot(x, y, label='Sine Function', color='blue', linestyle='-', linewidth=2) plt.xlabel('X-axis') plt.ylabel('Y-axis') plt.title('Simple Line Plot') plt.legend() plt.show()</pre>	K3
3.	Write python code to plot sine and cos wave. <pre>import matplotlib.pyplot as plt import numpy as np plt.title("Sine and Cos Curves") x = np.linspace(0, 10, 1000) plt.plot(x, np.sin(x), '-g', label = 'sin(x)') plt.plot(x, np.cos(x), ':r', label = 'cos(x)') plt.legend() plt.show()</pre>	K3
4.	How can you set different colors for a line plot? In Matplotlib, we can set different colors for a line plot using the color parameter in the plt.plot() function. The color parameter allows us to specify the color of the line in various ways. Eg. plt.plot(x, y, color='blue') # specify color by name plt.plot(x, y, color='g') # short color code (rgbcmyk) plt.plot(x, y, color='0.75') # Grayscale between 0 and 1	K3

5.	What functions can be used to draw scatter plots? Scatter plots can be produced using the 2 methods: plt.plot() and plt.scatter() Eg. plt.plot(x, y, 'o', color='black') plt.scatter(x, y, marker='o')	K2
6.	Compare plot and scatter functions. - plt.scatter() has the capability to render a different size and/or color for each point, so the renderer must do the extra work of constructing each point individually - plt.plot() - the points are always essentially clones of each other, so the work of determining the appearance of the points is done only once for the entire set of data. - plt.plot() function is noticeably more efficient and it should be preferred over plt.scatter() for large datasets.	K2
7.	What is the purpose of visualizing errors? How is it done? In visualization of data and results, showing these errors effectively can make a plot convey much more complete information. It is done depending on the nature of the errors and the context of the analysis. Error bars are a standard way to represent the variability or uncertainty associated with each data point in a plot. They are created using errorbar() function. Eg. plt.errorbar(x, y, yerr=dy, fmt='k')	K3
8.	Write a note on contour plots. Three-dimensional data can be displayed in two dimensions using contours or color-coded regions. It displays the variations in a dataset as contours or lines of constant values. Contour plots are commonly used in scientific and engineering fields to visualize relationships between three variables. There are three Matplotlib functions that can be helpful for this task: plt.contour for contour plots, plt.contourf for filled contour plots, and plt.imshow for showing images. Applications: Topographic Maps, Engineering and Science	K2
9.	List down the functions used with draw contour plots. - plt.contour for contour plots, - plt.contourf for filled contour plots, and - plt.imshow for showing images.	K1
10.	What is the purpose of histograms? Write a simple Python code to plot a histogram. Histograms are used to visualize the distribution of a dataset, showing the frequency or probability of different ranges or bins of values. They provide insights into the underlying patterns, central tendency, and spread of the data. <u>Python code:</u> <pre>import matplotlib.pyplot as plt import numpy as np plt.style.use('classic') data = np.random.randn(1000) plt.hist(data, bins = 10, color = 'green') plt.show()</pre>	K3

11.	What is np.meshgrid() function? np.meshgrid() is a NumPy function that generates coordinate matrices from one-dimensional arrays. It is often used for creating 2D grids that can be used for evaluating functions on a grid. The resulting matrices can be used as input to functions that operate on 2D data, such as plotting functions. Eg. x = np.array([1, 2, 3]) y = np.array([4, 5, 6]) X, Y = np.meshgrid(x, y) print(X, Y) <u>Output:</u> [[1 2 3] [1 2 3] [1 2 3]] [[4 4 4] [5 5 5] [6 6 6]]	K2
12.	What are plot legends? How can they be customized? Plot legends give meaning to a visualization, assigning labels to the various plot elements. They can be created using plt.legend() command. Customizing plot legends is possible in following aspects: • The location of the plot legend on the plot can be specified • The frame can be turned off • The shape of the frame can be used as rounded and shadowed • The vertical space between legend entries can be altered • A title can be provided for the legend. Eg. plt.legend(loc = 'upper left', frameon = False, fancybox = True, shadow = True, labelspace = 1, title = 'City Area')	K2
13.	What is colormap? A colormap or cmap is a set of colors used to represent the values in a plot or visualization. In data visualization, colormaps are applied to data to map numerical or categorical values to specific colors. Colormaps play a crucial role in conveying information in a visually effective way, especially in heatmaps, contour plots, surface plots, and other color-mapped visualizations. Few colormaps available are viridis, plasma, inferno, magma, cividis, jet, coolwarm, greys, rainbow, Blues, Reds	K2
14.	Mention the significance of subplots. Create a 2×3 grid of subplots using a Python function. Subplots are groups of smaller axes that can exist together within a single figure. They are a powerful tool in data visualization, offering a way to organize, compare, and present information in a structured and visually appealing manner. To create a 2×3 grid of subplots, where all axes in the same row share their y-axis scale, and all axes in the same column share their x-axis scale: fig, ax = plt.subplots(2, 3, sharex='col', sharey='row')	K3

15.	Why is annotation used in a plot? Give an example annotation statement. It is often useful to annotate certain features of the plot to draw the reader's attention. This can be done manually with the <code>plt.text</code> (or <code>ax.text</code>) command, which will place text at a particular x or y value. The <code>ax.text</code> method takes an x position, a y position, a string, and then optional keywords specifying the color, size, style, alignment, and other properties of the text. Eg. <code>plt.text('1-1-2024', 3950, 'New Year's Day', size=10, color='gray')</code>	K2
16.	Write a code to draw a three-dimensional line and scatter plot. # Three-dimensional scattered line <pre>ax = plt.axes(projection='3d') z = np.linspace(0, 15, 1000) x = np.sin(zline) y = np.cos(zline) ax.plot3D(x, y, z, 'gray')</pre> # Three-dimensional scattered points <pre>ax = plt.axes(projection='3d') zdata = 15 * np.random.random(100) xdata = np.sin(zdata) + 0.1 * np.random.randn(100) ydata = np.cos(zdata) + 0.1 * np.random.randn(100) ax.scatter3D(xdata, ydata, zdata, c=zdata, cmap='Greens');</pre>	K3
17.	What is a wireframe plot? How to create a 3-D wireframe plot? A wireframe plot is a type of 3D plot that represents a three-dimensional surface using lines. Instead of a filled surface, a wireframe plot displays the surface using a mesh of lines connecting the data points. This can be particularly useful for visualizing the structure of a surface or function in three-dimensional space. The <code>plot_wireframe()</code> function is commonly used to create wireframe plots.	K2
18.	Define surface plot. Write about plot_surface function. A surface plot is like a wireframe plot, but each face of the wireframe is a filled polygon. Adding a colormap to the filled polygons can aid perception of the topology of the surface being visualized. <code>plot_surface</code> - function is used in Matplotlib to create a 3D surface plot. It has the following parameters: • X, Y, and Z coordinates, • <code>rstride</code> (the step size between rows when plotting the surface) • <code>cstride</code> (the step size between columns when plotting the surface) • <code>cmap</code> (colormap to be used for coloring the surface) • <code>edgecolor</code> (the color of the edges of the surface plot) <u>Eg. code using <code>plot_surface()</code>:</u> <pre>ax = plt.axes(projection='3d') ax.plot_surface(X, Y, Z, rstride=1, cstride=1, cmap='viridis', edgecolor='none')</pre>	K2
19.	Brief on basemap tool kit. The Basemap toolkit in Matplotlib, often referred to as <code>mpl_toolkits.basemap</code>, provides a collection of tools for creating map plots in Python. It allows users to create various types of maps, including world maps, regional maps, and custom projections. The Basemap toolkit is	K2

	particularly useful for visualizing geospatial data in scientific, environmental, or geographical applications. Basemap toolkit is not included in the default Matplotlib installation. To use it, it needs to be separately installed.	
20.	What is seaborn? Seaborn is a Python data visualization library based on Matplotlib. It provides a high-level interface for creating informative and attractive statistical graphics. It provides High-Level Interface, Integration with Pandas, Statistical Visualizations, Regression and categorical Plots	K1

PART B

S. No.	Question	BTL																										
1.	Briefly explain about the line plot and scatter plot.	K2																										
2.	Explain the functions with contour plot and histograms.	K2																										
3.	Explore visualizing errors. Give examples and graphs that visualize errors.	K3																										
4.	Write short notes on the following: (i) Plot legends (ii) Colorbars (iii) Text and annotation in plots	K2																										
5.	Write a Python code to generate a line plot, a 2-D scatter plot and a 1-D histogram for the following data and include comments to explain the purpose of each component in your code: <table><tr><th>Months</th><th>Sales</th></tr><tr><td>Jan</td><td>120</td></tr><tr><td>Feb</td><td>150</td></tr><tr><td>Mar</td><td>200</td></tr><tr><td>Apr</td><td>180</td></tr><tr><td>May</td><td>250</td></tr><tr><td>Jun</td><td>300</td></tr><tr><td>Jul</td><td>280</td></tr><tr><td>Aug</td><td>320</td></tr><tr><td>Sep</td><td>200</td></tr><tr><td>Oct</td><td>180</td></tr><tr><td>Nov</td><td>210</td></tr><tr><td>Dec</td><td>240</td></tr></table>	Months	Sales	Jan	120	Feb	150	Mar	200	Apr	180	May	250	Jun	300	Jul	280	Aug	320	Sep	200	Oct	180	Nov	210	Dec	240	K3
Months	Sales																											
Jan	120																											
Feb	150																											
Mar	200																											
Apr	180																											
May	250																											
Jun	300																											
Jul	280																											
Aug	320																											
Sep	200																											
Oct	180																											
Nov	210																											
Dec	240																											
6.	How can multiple subplots be created? Explain with relevant examples.	K2																										
7.	What is 3D plotting? Explain the various concepts for 3D plotting with examples.	K2																										
8.	How graphical data can be projected using matplotlib? Explain with examples.	K2																										